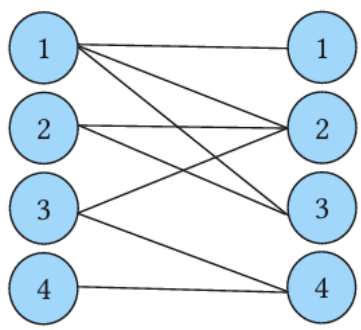
Problem 1. Suppose there are n members in a team, and there are n tasks. Each member can only perform a subset of the tasks (including possibly all, or none). All n tasks can be done independently of each other.

If each member can only perform at most one task, and each task can only be performed by at most one person, what is the maximum number of tasks that can be performed?

can perform?	task 1	task 2	task 3	task 4
member 1	yes	yes	yes	no
member 2	no	yes	yes	no
member 3	no	yes	no	yes
member 4	no	no	no	yes

The input to this problem is an $n \times n$ grid M, where row i and column j is true if member i can perform task j, and false otherwise.

We can model this with a graph G, where the nodes are members and tasks, and there is an edge between a member and a task if that member is able to do that task:



Exercise 2. Is M the adjacency matrix of G? Why or why not?

No, M is not the adjacency matrix of G, as it only describes connections from members to tasks. Since the graph is bipartite, the real adjacency matrix would go:

$$\begin{matrix} 0 & M \\ M^T & 0 \end{matrix}$$

for all values. M is in fact, the **biadjacency** matrix, which shows the connections between the two partitions. An actual adjacency matrix must include all nodes and all edges, so it's bigger and symmetric.

Exercise 4. Prove that the maximum matching is at most <= n/2.

In a graph G = (V,E) with |V| = n, a matching is a set of edges with no shared endpoints.

- Each edge in the matching uses 2 distinct vertices.
- If the matching has size k, it covers 2k distinct vertices.
- Since there are only n vertices total, we must have 2k <= n.

k <= n/2. So the size of any matching, and hence, the maximum matching, is at most n/2.

Exercise 5. Let G be an undirected graph. Prove that G is bipartite if and only if G is 2-colorable. (Hint: "Just color.")

[=>] "If G is bipartite, then G is 2-colorable."

Since G = (V,E) is bipartite, it has two parts L and R, where each edge in E connects a node in L to a node in R. We color all nodes in the set L red, and all nodes in the set R blue. This is valid since no node in L is adjacent to any other node in L, and no node in R is adjacent to any other node in R. Thus, G is 2-colorable.

[<=] "If G is 2-colorable, then G is bipartite."

Since G = (V,E) is 2-colorable, it is possible to color each node one of two colors so that no two adjacent nodes have the same color. Suppose that we've colored all nodes either red or black. We partition the two types of nodes into two sets: L is the set of red nodes, while R is the set of black nodes. Then, the graph is bipartite because there must be an edge between a red node and a black node for it to be different colors, and no edges between nodes of the same color.

Exercise 6. Find a linear-time algorithm that 2-colors an undirected graph, or determines if there is no 2-coloring.

We run BFS from an arbitrary unvisited vertex:

- Color the starting vertex red.
- When exploring a vertex u, color all its unvisited neighbors the opposite color of u, say blue.
- If a neighbor is already visited and has the same color as u, the graph is not 2-colorable (return False).

Repeat for each connected component, if no conflict is found, return the 2-coloring.

Exercise 8. Explain why any edge in E(L,R) can be assigned any capacity in [1, inf], without changing the maximum flow.

Any node in E(L,R) can be assigned any capacity in [1, inf] without changing the maximum flow because the flow through any such edge e in E(L, R) is inherently limited by the capacity of the incoming edge from s to the left node i in L.

Since the edge s -> i has capacity 1, the total flow entering is at most 1. By conservation of flow, the total flow leaving i, which includes the flow through e, is also at most 1. Even if the capacity of e is set to any value greater than or equal to 1, the actual flow through e can never exceed 1, and changing the capacity of e within [1, inf] will have no effect on the maximum flow.

Exercise 9. Prove that the maximum matching is equal to the max flow in the flow network above.

We construct a flow network from the bipartite graph G = (L ∪ R, E):

- Add source s connected to each u in L with capacity 1
- Add sink t connected to each v in R with capacity 1
- Orient original edges L -> R with capacity [1, inf]

An integral flow of value k corresponds to k units of flow from s to t, each passing through a unique u in L and a unique v in R, using distinct vertices (due to unit capacities on s -> L and R -> t). Thus, it defines a matching of size k. Conversely, any matching of size k gives a feasible integral flow of value k. By the integral flow theorem, max flows equals max matching.

Exercise 10. Find an $\mathcal{O}(n + e)$ algorithm that extracts a maximum matching given a max flow.

Given a max flow of value k in the constructed network:

1. For each edge (u,v) with u in L, v in R:
    - If the flow on (u,v) is 1, add (u,v) to the matching.
2. Each u in L has at most one outgoing flow unit due to capacity 1 on s->u, and each v in R has at most one incoming flow unit, due to capacity 1 on v->t, so the selected edges form a valid matching of size k.

It takes O(n + e) time to scan all the edges.

Exercise 11. Prove that a <= min(|L|, |R|) <= n/2. (Hint: Use the integral flow theorem, Use Exercise 4).

Let *a* be the number of augmentations in the Ford-Fulkerson algorithm on the bipartite matching flow network.

By the integral flow theorem, each augmentation increases the flow value by exactly 1. The maximum possible flow value is at most min(|L|, |R|), since flow must pass through distinct vertices in L and R due to unit capacities from s and to t.

From Exercise 4, the maximum matching size is <= n/2. Since |L| + |R| = n, we have min(|L|, |R|) <= n/2.

Thus, a <= min(|L|, |R|) <= n/2.

Exercise 13. Prove that a perfect matching is a maximum matching.

A perfect matching is a matching of size exactly n/2. By Exercise 4, any matching has size at most n/2. Thus, a perfect matching achieves this upper bound, so it must be a maximum matching.

Exercise 14. Prove that a bipartite graph G = (L ⊔ R, E) can only have a perfect matching if |L| = |R|.

idea: perfect matching needs n/2, since |L| + |R| = n, then to match them all exactly, we need them both n/2 each

Let G = (L ⊔ R, E) be a bipartite graph with |L| + |R| = n. A perfect matching matches every vertex exactly once, so it must cover all n vertices.

Each edge in the matching pairs one vertex from L with one from R. Thus, all |L| vertices in L are matched to distinct vertices in R, so |L| <= |R|. Similarly, all |R| vertices in R are matched to distinct vertices in L, so |R| <= |L|.

Therefore, |L| = |R| = n/2.

Supplementary Question: Let i, j ∈ L be distinct nodes on the left. Suppose i and j both have degree 1, and are adjacent to the exact same node k ∈ R. Then there can't be any perfect matching. Why?

Because only one of the nodes from L can match with R, with the other unpicked node not being matched with anyone since the only node its adjacent to is already taken.

Exercise 16. Prove that a graph is bipartite if and only if it has no odd cycles.

[=>] “If a graph is bipartite, then there is no odd cycle in it.”

In a bipartite graph, vertices alternate between the two parts along any walk. A cycle must return to the starting vertex after an even number of steps, so it must have an even length. Hence, n odd cycle exists.

[<=] “If a graph has no odd cycles, then it is bipartite.”

We assume that G has no odd cycles, and pick a connected component. We choose an arbitrary vertex v, and color it red.

We run BFS as we did in Exercise 10, assigning each vertex the opposite color of its parent. This ensures that along any path from v, vertices alternate colors.

We consider any edge (x,y):

[Case 1: x and y are in different BFS levels] Then they have opposite colors by construction, showing that they are bipartite so far.

[Case 2: x and y are in the same BFS level] Let d(v,x) = k and d(v,y) = k. Then they have paths: v->...->x and v->...->y, both of length k. Appending edge x->y to that cycle, we have a path of length k + k + 1 = 2k + 1, which is the total length of the cycle. However, this is a contradiction since we assumed that there are no odd cycles.

Thus, no edge connects two vertices of the same color. So therefore, the coloring is valid, and G is bipartite.

Exercise 18. Let MIN-VERTEX-COVER be the (function) problem of determining the size of the minimum vertex cover in an undirected graph. Prove that MIN-VERTEX-COVER is NP-hard. (Hint: Reduce VERTEX-COVER to it.)

VERTEX-COVER (decision version): Given graph H and integer k, does G have a vertex cover of size <= k? This is known to be NP-complete.

We reduce VERTEX-COVER to MIN-VERTEX-COVER. Given an instance (G, k) of VERTEX-COVER, run MIN-VERTEX-COVER on G to get the minimum cover size c. Then answer YES if c <= k, NO otherwise.

Exercise 19. Find an O(ne) time algorithm that determines the size of the maximum independent set in a bipartite graph.

For a bipartite graph G (L ⊔ R, E):

1. Find a maximum matching M (e.g., using Kuhn's algorithm in O(ne) time).
2. By Konig's theorem, the size of the minimum vertex cover is |M|.
3. In any graph, the complement of a vertex cover is an independent set:
    a. Independent set size = n - vertex cover size
4. Thus, the maximum independent set size = n - |M|.

This takes O(ne) for maximum matching, and O(1) computation.

Exercise 22. Prove the Weak form of Kőnig's theorem. In a bipartite graph, the size of the maximum matching is at most the size of the minimum vertex cover.

Let M be any matching and C be any vertex cover. Every edge in M must be covered by C. Since edges in M are disjoint, meaning they don't share endpoints, each edge requires at least one distinct vertex from C to cover it. Thus |M| <= |C|. Taking M as a maximum matching and C as a minimum vertex cover gives the result.

Exercise 23. Prove that if M is a matching and C is a vertex cover in a bipartite graph such that |M| = |C|, then M is a maximum matching and C is a minimum vertex cover.

Let M* be a maximum matching and C* be a minimum vertex cover. By Exercise 22, |M| <= |M*| <= |C*| <= |C|. Given |M| = |C|, all inequalities become equalities. Thus, |M| = |*M|, and |C| = |C*|, therefore C is minimum.

Exercise 29. Find an O(n^2) implementation of the Gale-Shapley algorithm.
see [exercise29.py]

Exercise 32. Prove using Claim 30 that the Gale-Shapley algorithm returns a unique stable matching for any given set of preferences.

Claim 30 states that each woman (proposer) ends up with her best possible partner among all stable matchings.

If the algorithm could return two different stable matchings S_1 and S_2 for the same preferences, then some woman *w* would have different partners in S_1 and S_2. But by Claim 30, her partner in any stable matching must be the same best possible partner, hence a contradiction.

Therefore, the Gale-Shapley algorithm returns a unique stable matching.

Exercise 33. Find an input instance to the stable matching problem where there is more than one stable matching:

n = 2
where the preferences are:
woman 1: A > B
woman 2: B > A
man A: 2 > 1
man B: 1 > 2

Hence, we have the two stable matchings:

1. (1,A), (2,B) (where the women are the proposers)
2. (1,B), (2,A) (where the men are the proposers)

Note that the Gale-Shapley algorithm gives advantages to the one who proposes. But both arrangements are stable.

Exercise 35. Find an input instance where Problem 34, the Stable Roommates Problem, where the proposers is indistinguishable from the proposees, doesn't have a stable solution.

Let n = 2, where there are 4 students. Let students be A,B,C,D. Preferences:
A: B > C > D
B: C > A > D
C: A > D > V
D: Any

Here, A, B, C form an odd cycle of preferences. A prefers B over C, B prefers C over A, C prefers A over B. In any pairing, one of these three will be matched with D, leaving the other two

together. But those two will always prefer each other over their current partner, creating a blocking pair. Thus, no stable matching exists.